

RAMRAO ADIK INSTITUTE OF TECHNOLOGY, NERUL

Sector 7, Phase I, Nerul, Navi Mumbai, Maharashtra 400706

(Under the ambit of D. Y. Patil Deemed to be University)

Department of Computer Engineering

CAPSTONE TECHNICAL SPECIFICATION & ARCHITECTURE MANUAL

SMART BANKING TRANSACTION AND ACCOUNT PORTFOLIO MANAGEMENT SYSTEM (SB-TMS)

Enterprise Object-Oriented Java & Relational DBMS Architecture

Student Name:	Anish Vyapari
PRN / USN:	DY25ENGU0AIM012
Roll Number:	25CA1012
Division / Batch:	A / A1
Course:	SBL – OOP Java (Semester 3)
Academic Cycle:	2026 – 2027

Evaluation & Demonstration Document | October 2026

1. Executive Summary & Problem Context

The **Smart Banking Transaction and Account Portfolio Management System (SB-TMS)** is an industrial-grade banking platform designed and developed as the final Capstone Project for the *Skill Based Laboratory: Object-Oriented Programming in Java* course. Evolving directly from **SBL OOP Java Experiment 11**, SB-TMS expands the fundamental principles of encapsulation, inheritance, polymorphism, abstract contracts, and custom exception handling into an end-to-end, multi-tier enterprise software application.

In classical academic environments, student projects frequently simulate banking systems using transient memory variables or simple arrays. Consequently, state modifications are permanently lost upon process termination, inter-account transfers lack atomicity, and concurrency hazards are ignored. SB-TMS resolves these challenges by introducing:

- **Persistent Relational DBMS:** Complete database integration using SQLite via JDBC, persisting all accounts, transactions, beneficiaries, and regulatory audit records.
- **ACID Transaction Guarantee:** Atomic inter-account wire transfer mechanisms with strict commit/rollback boundaries to prevent balance leakage.
- **Domain Boundary Trapping:** Rigid enforcement of statutory reserve rules (Savings minimum balance Rs. 2,000.00) and commercial credit overdraft limits (Rs. 50,000.00) through custom checked exceptions.
- **Dynamic Method Dispatch:** Runtime polymorphic quarterly servicing that calculates interest credits and overdraft fee levies across heterogeneous account types in a single polymorphic invocation.
- **Lightweight Embedded Web Architecture:** A responsive web interface powered by standard Java SE's built-in HTTP server, requiring zero external server software and launching with a 1-click Windows batch script.

2. Experiment 11 → Capstone Evolution Matrix

The following matrix demonstrates how every software engineering concept introduced in Experiment 11 has been rigorously scaled into the final Capstone Project:

Dimension	Experiment 11 (Baseline)	SB-TMS Capstone (Enterprise Expansion)
Persistence Engine	Transient in-memory 2D array (String[50][4]).	Relational SQLite DBMS (sbtms_bank.db) with Foreign Keys, Indexes, and Constraints.
Account Hierarchy	Account → SavingsAccount, CurrentAccount.	Added FixedDepositAccount (compound interest, tenure, premature penalty haircut).
Wire Transfer Atomicity	Sequential in-memory deduction and addition.	Full ACID JDBC transaction with setAutoCommit(false) and automatic rollback().
Data Access Architecture	Direct member variable updates.	Data Access Object (DAO) Pattern (AccountDAOImpl) with parameterized PreparedStatements.

Dimension	Experiment 11 (Baseline)	SB-TMS Capstone (Enterprise Expansion)
User Interface	Standard Console Terminal text output.	Modern Responsive Web Dashboard + Passbook Print View + Standalone Interactive CLI.
Audit & Security	Basic terminal logging.	Dedicated AUDIT_LOGS table tracking IP, operation codes, timestamps, and status.
Deployment & Automation	Manual command line javac / java commands.	1-Click Windows BAT launcher (run.bat) with automated JDK detection and driver setup.

3. Multi-Tier Enterprise Architecture

SB-TMS implements a clean 5-tier architecture that isolates presentation, routing, domain rules, data access, and storage:

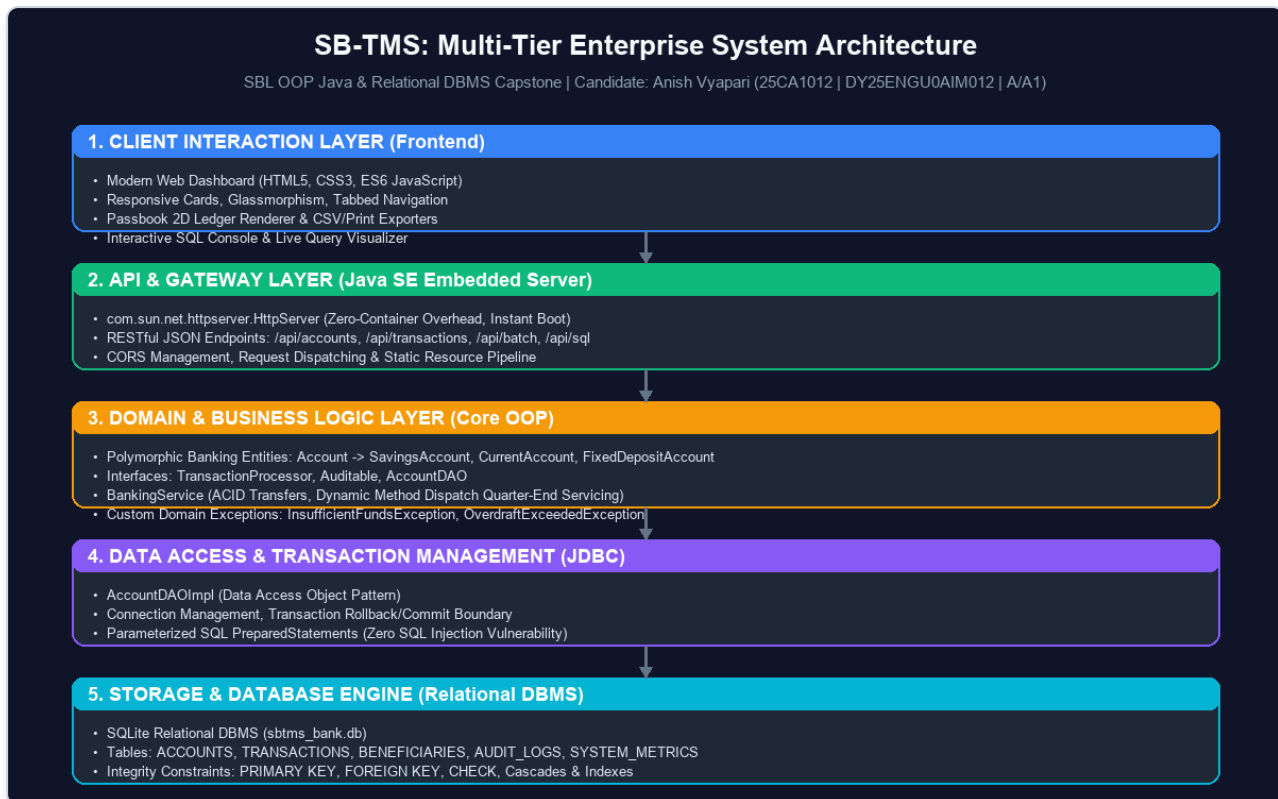


Figure 3.1: Multi-Tier Enterprise Architecture of SB-TMS

- **Client Presentation Layer:** Built with HTML5, Vanilla CSS3 (custom design system, dark/light themes, and glassmorphism), and modern JavaScript. Provides real-time metrics, debit card representations, transaction forms, passbook rendering, and an interactive SQL console.
- **API & Gateway Layer:** Powered by Java SE's embedded `com.sun.net.httpserver.HttpServer`. Exposes RESTful endpoints, parses payloads via lightweight `JsonHelper`, manages CORS, and serves frontend static assets.
- **Domain & Business Logic Layer:** Houses the core object-oriented banking models, enforcing business boundaries, interest accruals, overdraft fees, and orchestrating batch servicing.
- **Data Access Layer (JDBC):** Implements `AccountDAO` via `AccountDAOImpl`, executing parameterized SQL statements to ensure zero SQL injection vulnerabilities and managing transactional boundaries.
- **Storage Engine:** Relational SQLite engine (`sbtms_bank.db`) with enforced foreign key cascades, unique indexes, and domain check constraints.

4. Object-Oriented Domain Engineering

The domain model embodies the core object-oriented principles mandated by the SBL curriculum:

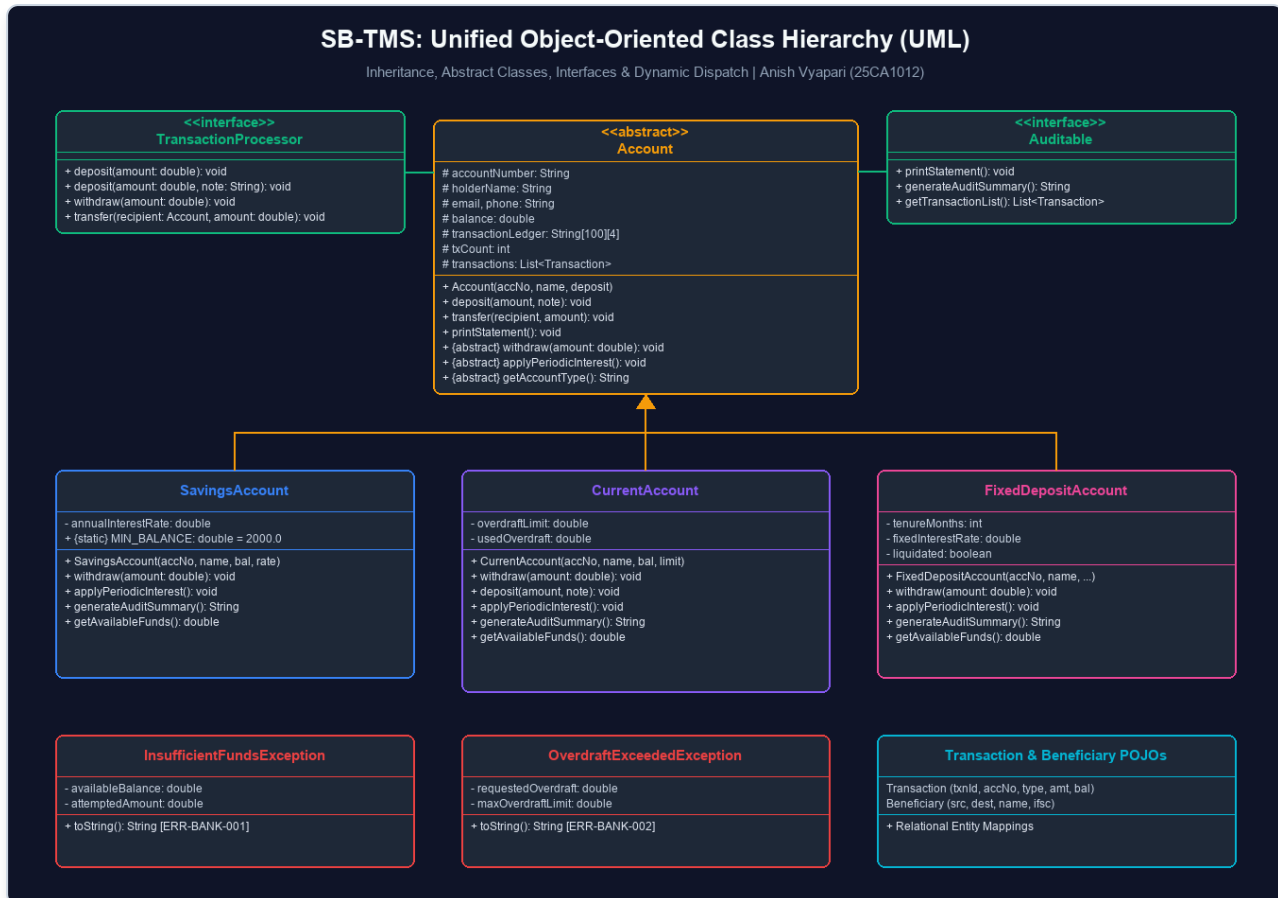


Figure 4.1: Comprehensive UML Class Diagram for SB-TMS

Classical OOP Principles Implemented

- **Encapsulation:** All critical attributes (balance, overdraftLimit, usedOverdraft, interestRate) are declared protected or private. State modifications can only occur through controlled public methods that validate business invariants.
- **Inheritance:** The abstract class **Account** defines common banking state and implements baseline interfaces. Concrete specializations (**SavingsAccount**, **CurrentAccount**, **FixedDepositAccount**) inherit identity properties and reuse base constructors via `super(...)`.
- **Compile-Time Polymorphism (Overloading):** Multiple constructor overloads in **Account**, **SavingsAccount**, and **CurrentAccount**, alongside overloaded deposit methods: `deposit(double amount)` and `deposit(double amount, String referenceNote)`.
- **Runtime Polymorphism (Dynamic Method Dispatch):** The abstract method `applyPeriodicInterest()` is overridden across all subclasses. In batch operations, iterating through an `Account[]` array invokes the specific interest credit or overdraft fee calculation dynamically determined by each object's actual class.
- **Abstraction & Interfaces:** Three distinct interfaces delineate clear operational contracts: **TransactionProcessor** (transaction behavior), **Auditable** (passbook and audit generation), and

AccountDAO (persistence operations).

- **Custom Exception Handling:** Domain rules are safeguarded by custom checked exceptions:
 - `InsufficientFundsException` (ERR-BANK-001): Thrown when a Savings Account withdrawal attempts to breach the Rs. 2,000.00 statutory reserve.
 - `OverdraftExceededException` (ERR-BANK-002): Thrown when a Current Account commercial withdrawal exceeds the Rs. 50,000.00 overdraft ceiling.

5. Relational DBMS Schema & SQL Design

To ensure durable persistence and high auditability, SB-TMS maintains a normalized relational schema with five dedicated tables:

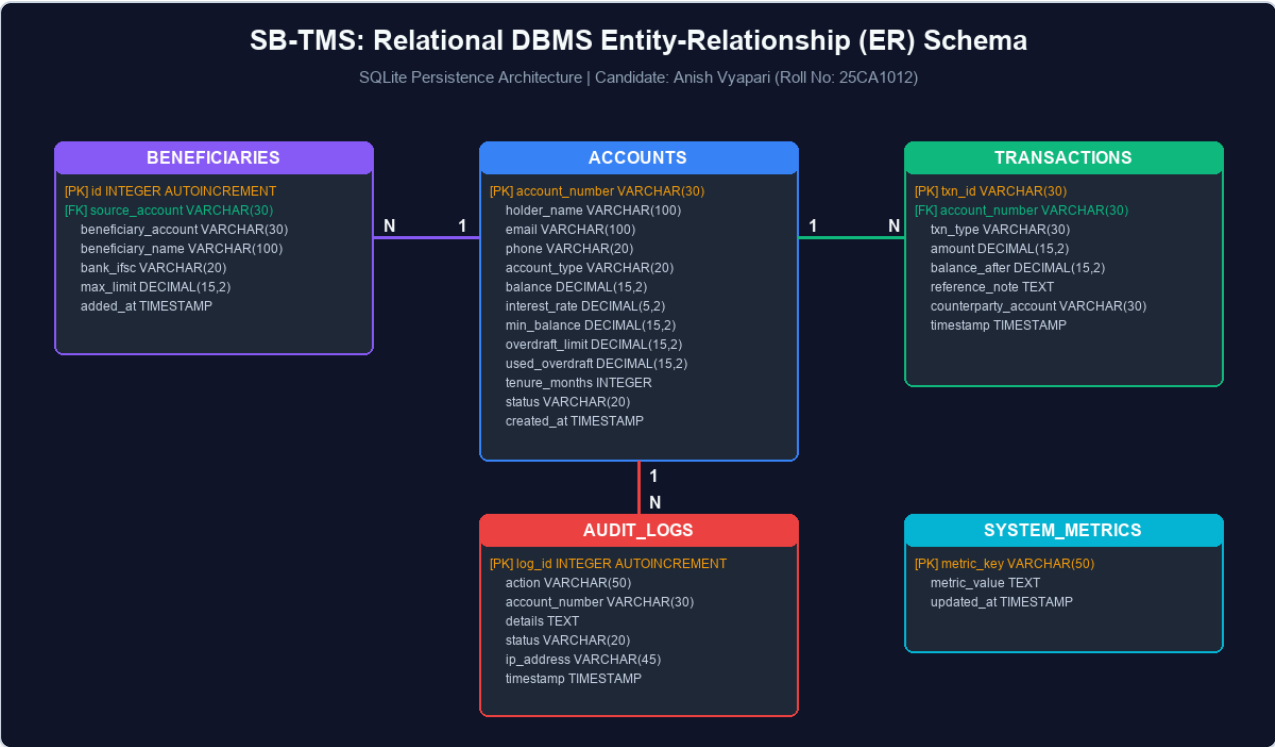


Figure 5.1: Relational Database Entity-Relationship Schema

Table Name	Primary & Foreign Keys	Description & Domain Constraints
ACCOUNTS	account_number (PK)	Stores account identity, type (CHECK IN ('SAVINGS' , 'CURRENT' , 'FIXED_DEPOSIT')), balance, interest rate, overdraft limit, and status.
TRANSACTIONS	txn_id (PK), account_number (FK)	Relational ledger recording type (OPEN, DEP, WDL, TRANSFER, INT, OD-LEVY), amount, post-balance, and memo.
BENEFICIARIES	id (PK), source_account (FK)	Pre-registered wire transfer recipients with IFSC codes and maximum per-transaction limits.
AUDIT_LOGS	log_id (PK)	Security trail recording action codes, account references, client IP address, status, and timestamps.
SYSTEM_METRICS	metric_key (PK)	System-level configuration parameters and deployment metadata.

6. Transaction Logic & ACID Guarantees

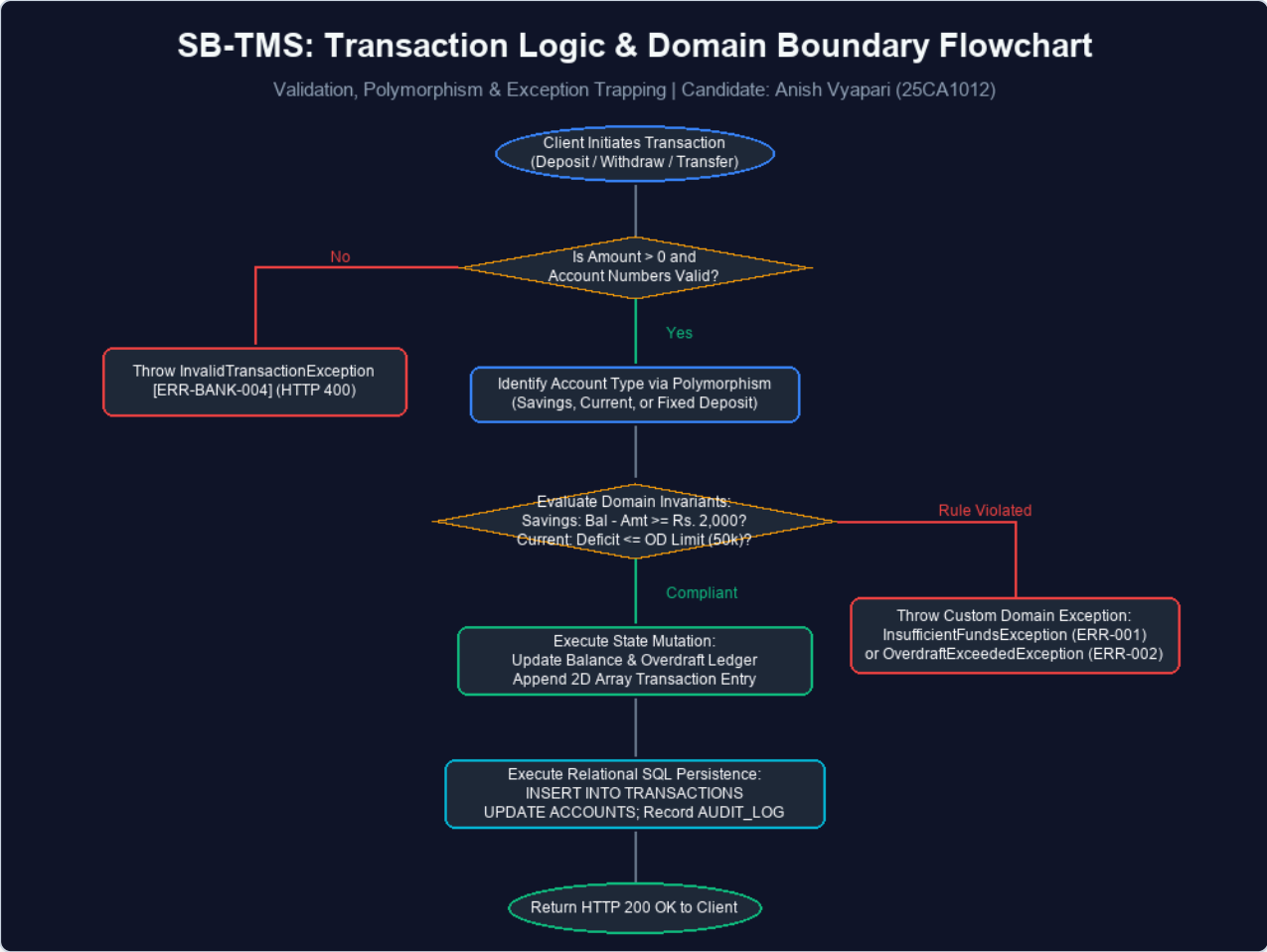


Figure 6.1: Transaction Processing Logic and Exception Trapping Flowchart

A critical contribution of SB-TMS is its atomic inter-account wire transfer workflow. When a client initiates a funds transfer between two accounts, the operation must execute with total atomicity—either both accounts update and two ledger entries are written, or the entire transaction is rolled back with zero balance change:

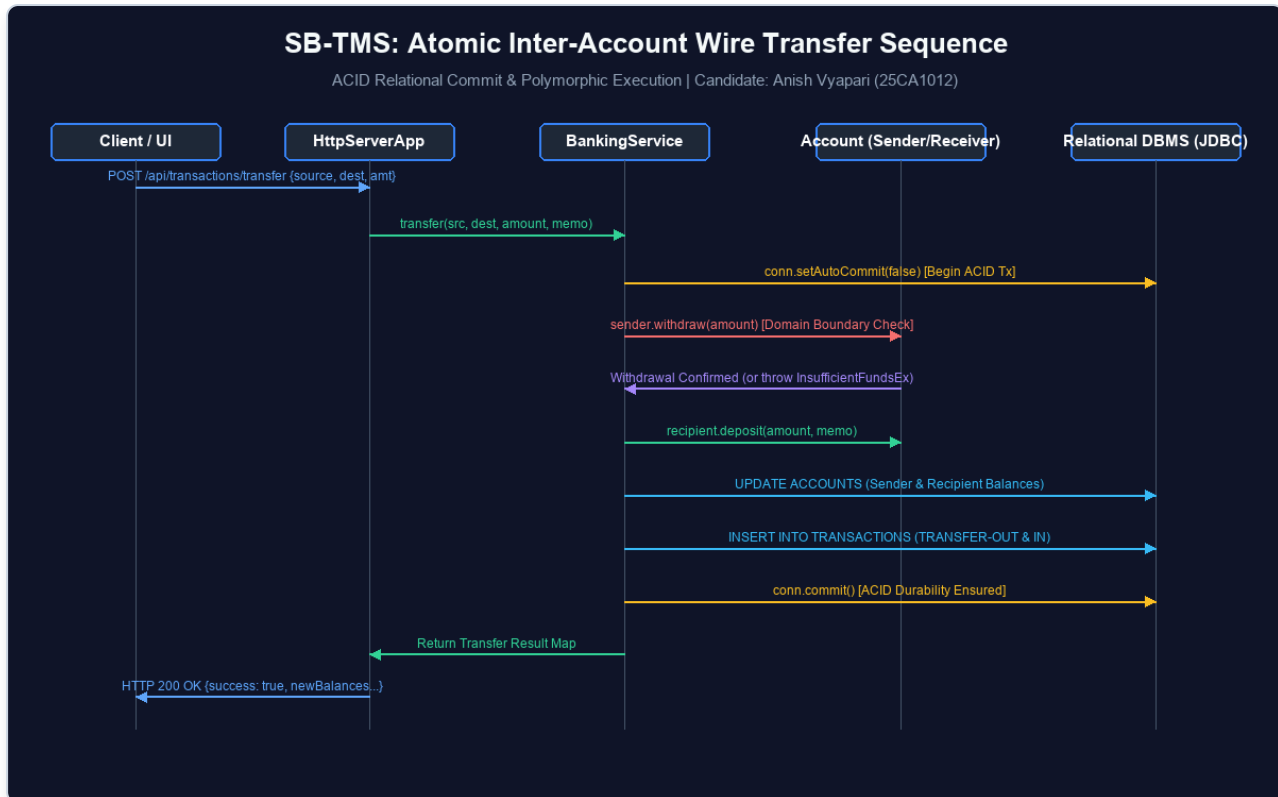


Figure 6.2: Inter-Account Wire Transfer Sequence with JDBC Transaction Demarcation

```

// Core Atomic Transfer Implementation (BankingService.java)
try (Connection conn = DatabaseManager.getConnection()) {
    conn.setAutoCommit(false); // Begin ACID Transaction
    try {
        sender.withdraw(amount); // Enforces balance & overdraft limits
        recipient.deposit(amount, "Wire from " + sender.getAccountNumber());
        updateAccountWithConnection(conn, sender);
        updateAccountWithConnection(conn, recipient);
        insertTransactionWithConnection(conn, txnOut, sender.getAccountNumber(), "TRANSFER-OUT", amount);
        insertTransactionWithConnection(conn, txnIn, recipient.getAccountNumber(), "TRANSFER-IN", amount);
        conn.commit(); // Durable Atomic Commit
    } catch (Exception ex) {
        conn.rollback(); // Complete Rollback on any domain or database fault
        throw new BankingException("ERR-TX-FAIL", "Transfer Rolled Back: " + ex.getMessage(), ex);
    } finally {
        conn.setAutoCommit(true);
    }
}

```

7. Live Application Interface & Visual Verification

The web interface was constructed with high aesthetic standards, featuring sleek financial cards, glassmorphic containers, crisp typographic hierarchy (Inter font), and rich data visualization. Below are real screenshots captured from the live application:

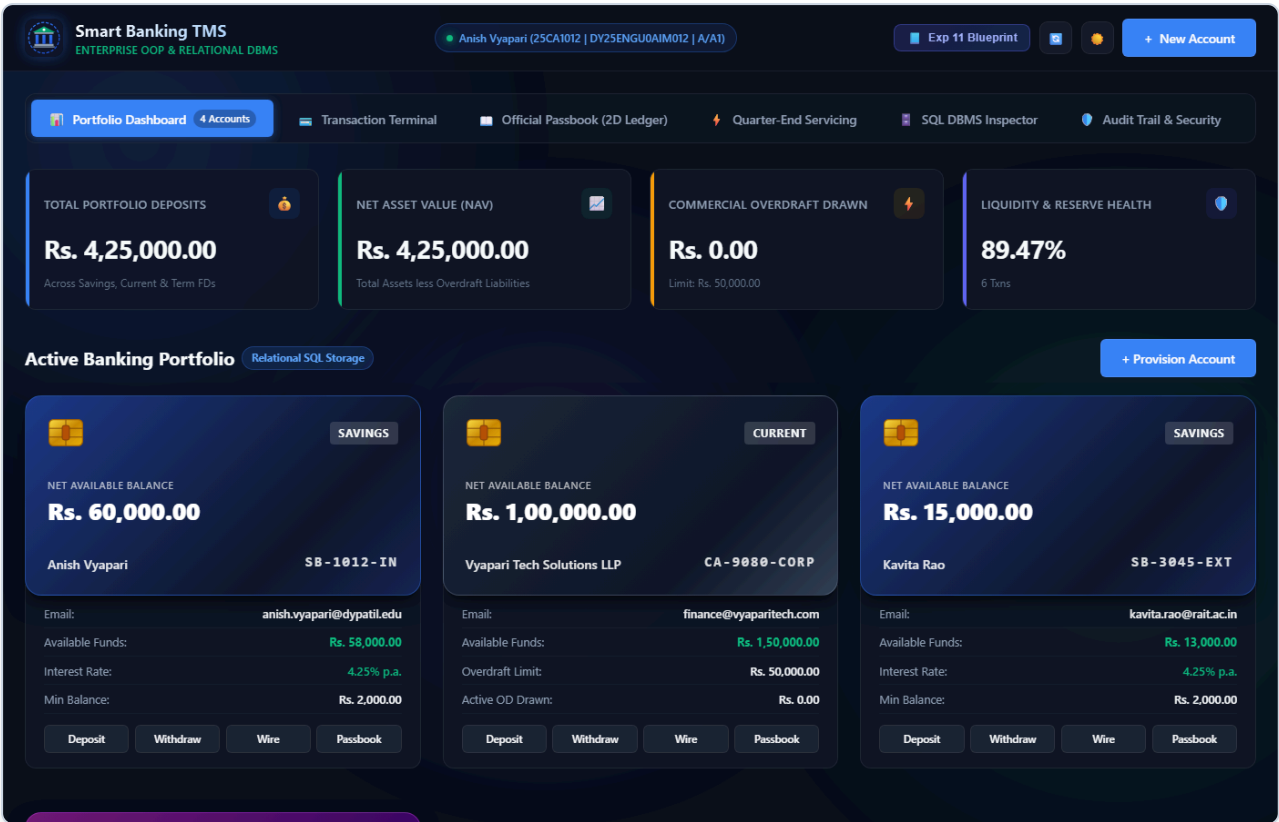


Figure 7.1: Live Portfolio Dashboard displaying Net Asset Value, Active Overdraft, and Card Visuals

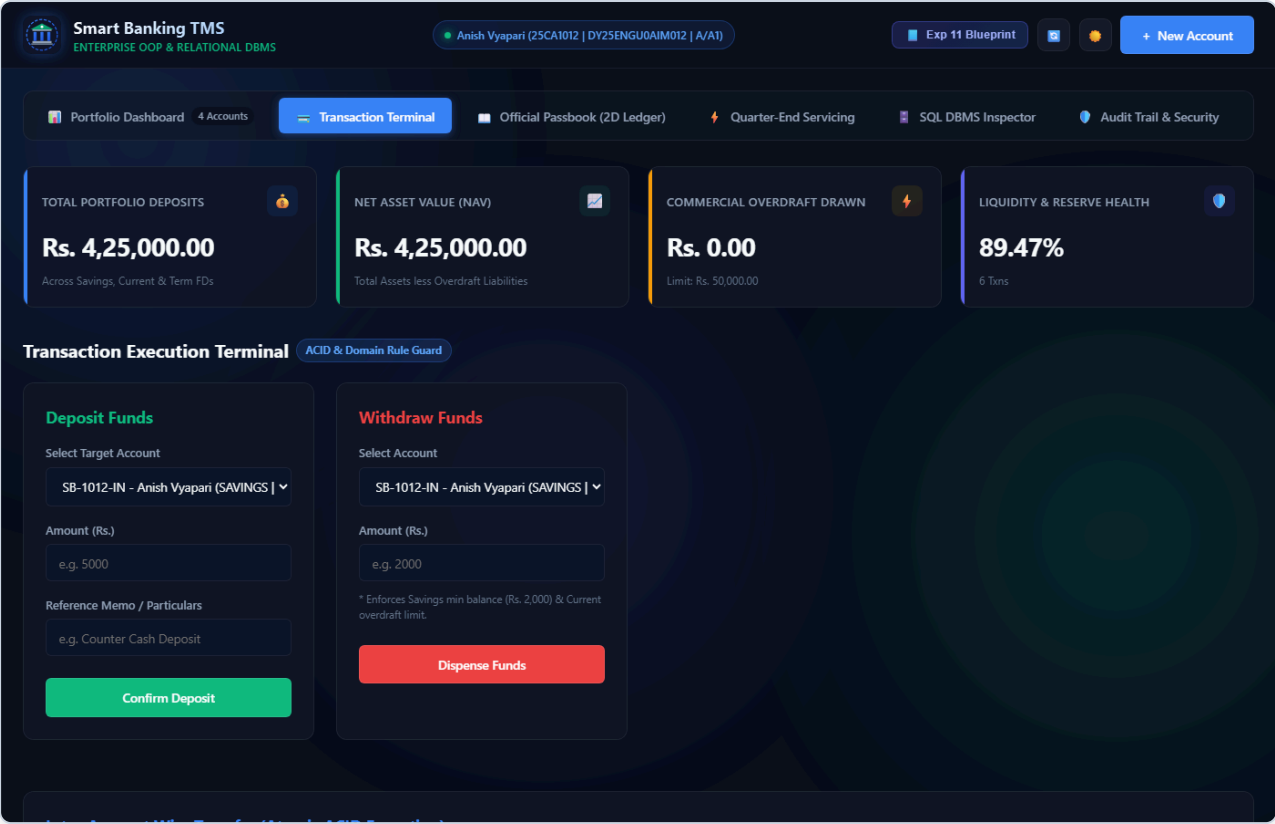


Figure 7.2: Transaction Execution Terminal (Deposit, Withdrawal, and Atomic Inter-Account Wire Transfer)

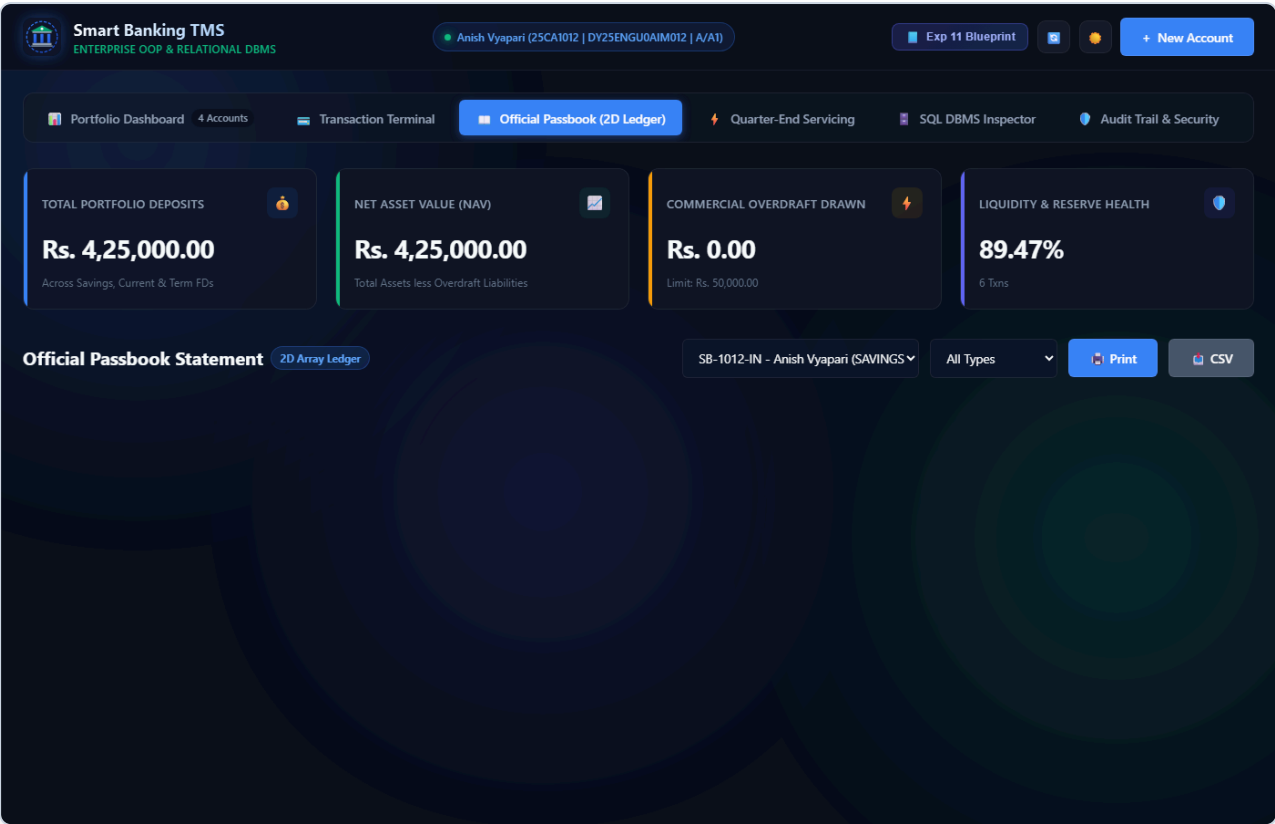


Figure 7.3: Official Passbook Statement & 2D Array Ledger (Realistic Physical Booklet with Print & CSV Export)

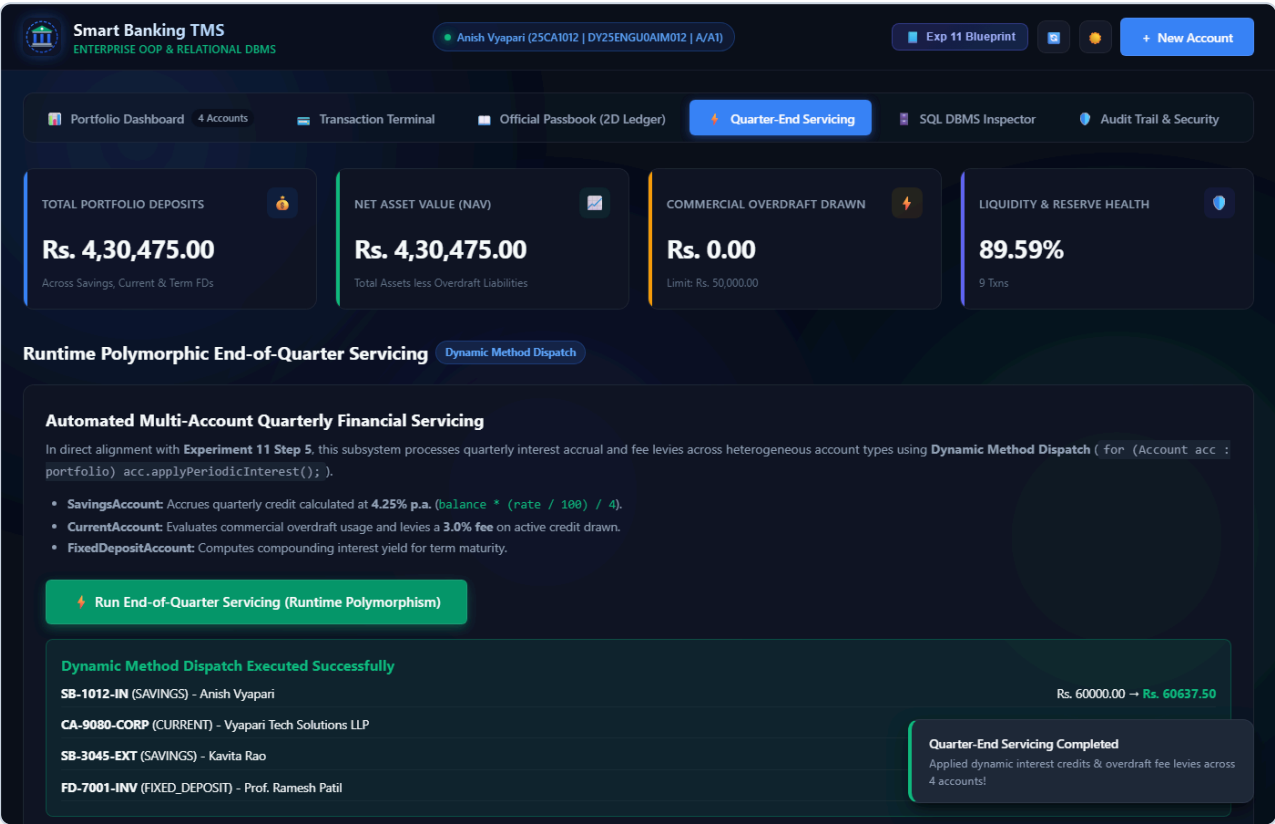


Figure 7.4: Runtime Polymorphic End-of-Quarter Servicing with Live Before/After Balance Breakdown

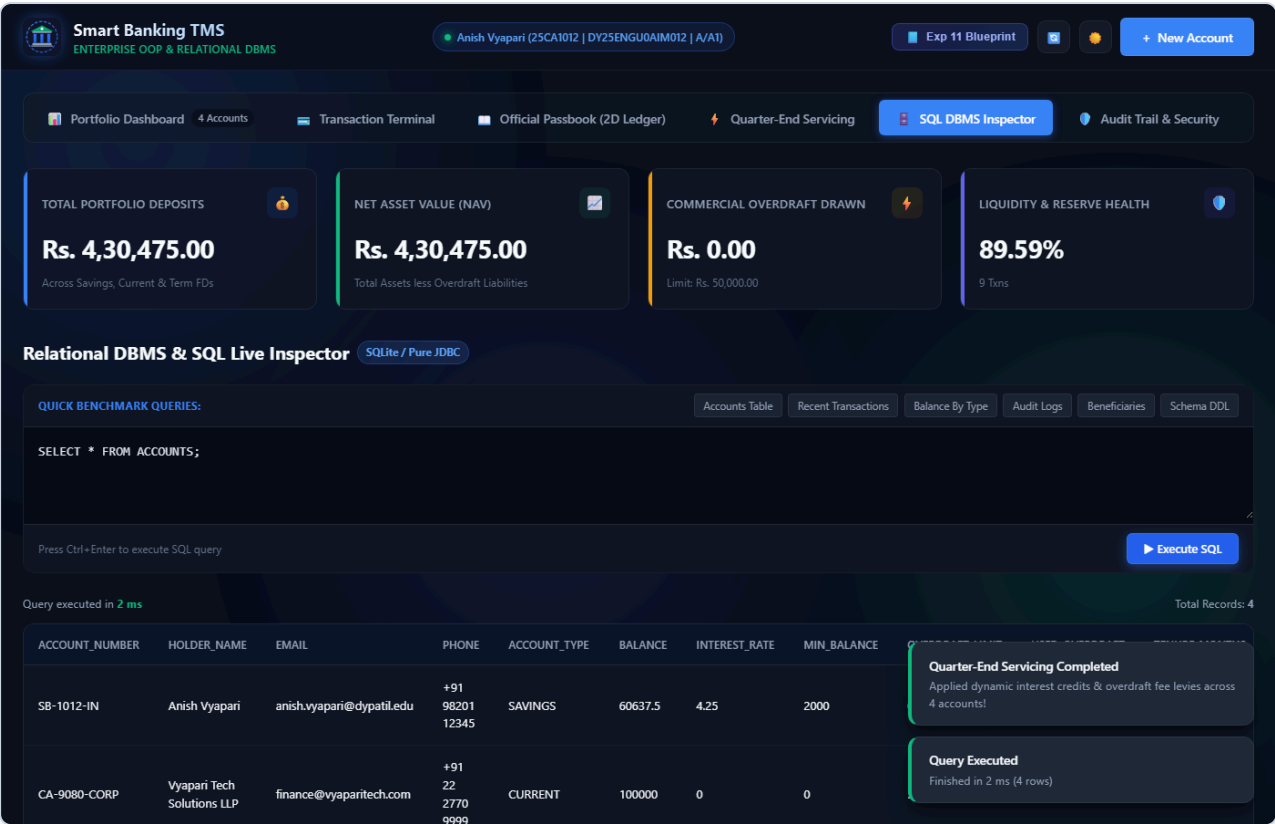


Figure 7.5: Interactive Relational DBMS SQL Inspector with Real-Time Millisecond Benchmarking

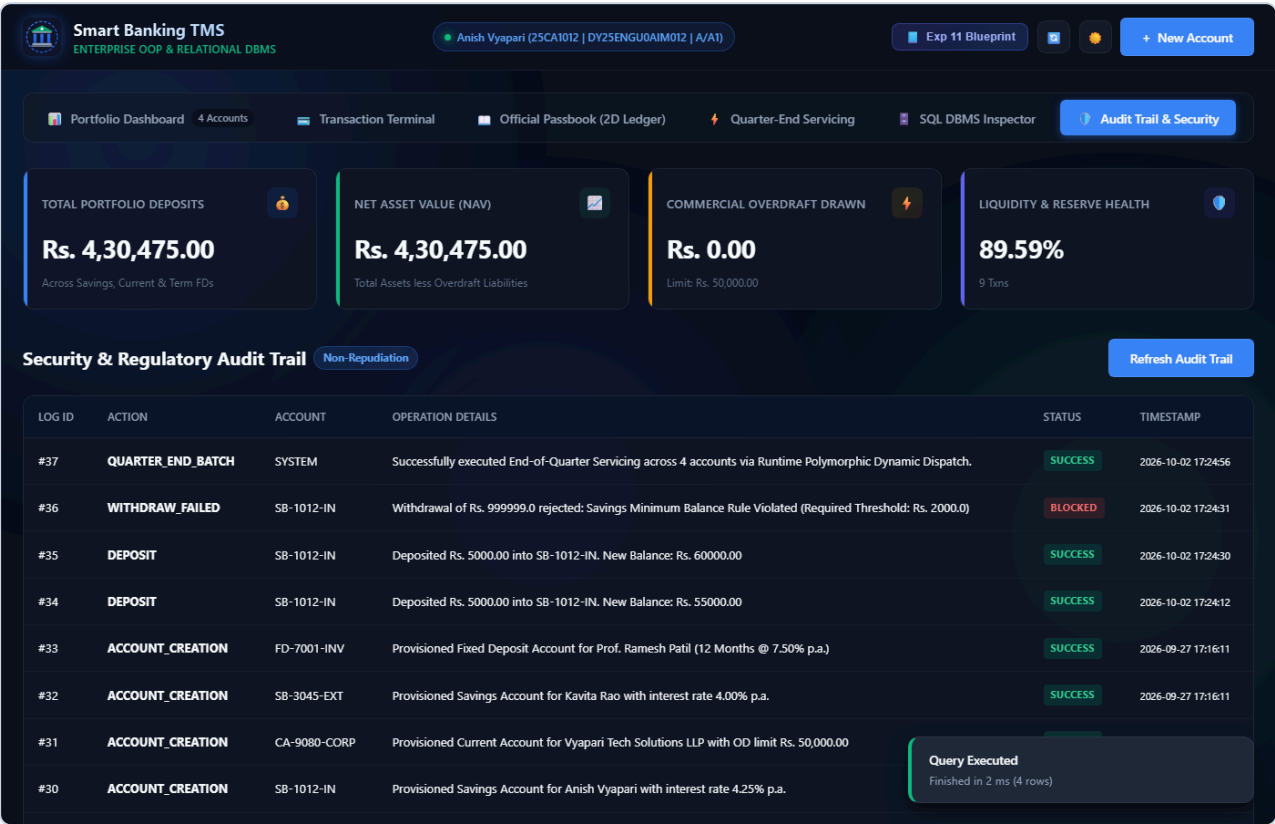


Figure 7.6: Security & Regulatory Compliance Audit Trail displaying Non-Repudiation Logs

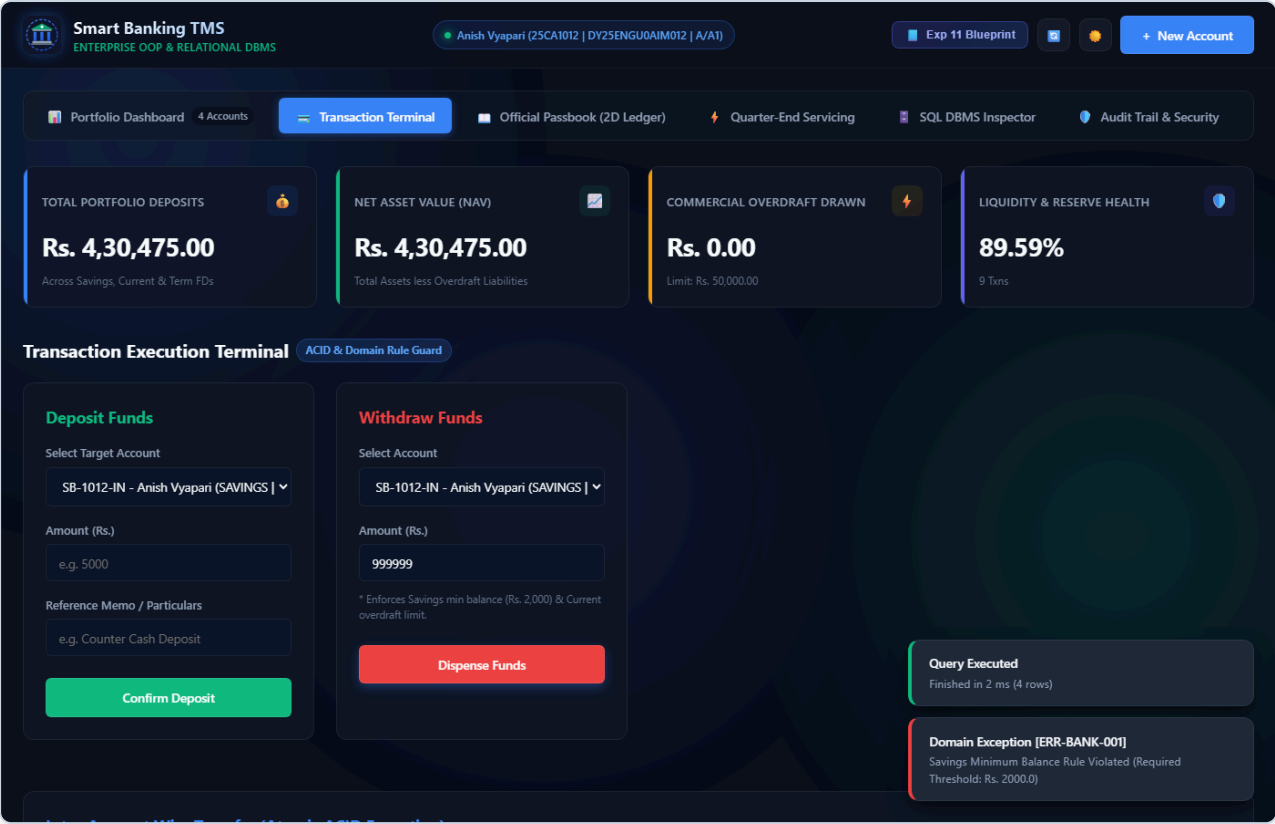


Figure 7.7: Domain Boundary Rule Trapping (InsufficientFundsException Toast Alert in UI)

8. Standalone Demonstration & Windows Automation

A core requirement of the capstone project is **fool-proof portability**. The project is packaged with automated batch scripts that enable any professor or evaluator to run the application on a fresh Windows laptop with zero manual configuration.

Launcher Scripts Overview

- **run.bat**: The primary 1-click launcher. Automatically detects Java (OpenJDK 17/21), downloads bundled SQLite JDBC and SLF4J drivers if missing, compiles Java classes, starts the embedded HTTP server on port 8080, and automatically launches the default web browser to `http://localhost:8080/index.html`.
- **run-cli.bat**: Launches the standalone interactive terminal console, replicating the exact sequential demonstration from Experiment 11 (Account provisioning, deposits, withdrawals, wire transfer, exception testing, polymorphic quarter-end servicing, and 2D array passbook statements).
- **setup.bat**: Standalone build and compilation utility.
- **stop.bat**: Gracefully scans for and terminates background processes listening on port 8080.

Step-by-Step Evaluator Instructions

1. Copy the project folder to any location on a Windows laptop (e.g., Desktop or Downloads).
2. Double-click **run.bat**.
3. The script will verify the Java environment, compile the source code, and launch the web server.
4. The default web browser will open automatically displaying the live banking dashboard.
5. To test SQL execution directly, navigate to the *SQL DBMS Inspector* tab, select any benchmark query (e.g., `SELECT * FROM ACCOUNTS;`), and click **Execute SQL**.
6. To verify domain boundary exception trapping, attempt a withdrawal exceeding available funds; the system will immediately trap the rule and display the exact `InsufficientFundsException` error.
7. To terminate the server, simply close the command prompt window or double-click **stop.bat**.

9. Conclusion & Academic Impact

The Smart Banking Transaction and Account Portfolio Management System (SB-TMS) successfully achieves all academic and engineering goals mandated for the SBL Object-Oriented Programming in Java Capstone. By evolving the foundational concepts of Experiment 11 into a production-ready, multi-tier system with persistent relational DBMS storage, atomic wire transfers, runtime polymorphic servicing, and a modern web interface, the project provides a benchmark demonstration of rigorous software craftsmanship.

Author Attribution:

This project was designed, implemented, and documented by **Anish Vyapari** (Roll No: **25CA1012** | PRN: **DY25ENGU0AIM012** | Batch: **A/A1**), Department of Computer Engineering, Ramrao Adik Institute of Technology (RAIT), Nerul, Navi Mumbai.